DESIGN PATTERN

BECSE307T

by
Ms. Titiksha Bhagat

Object Oriented software

- Designing OO software is hard
- Designing *reusable* OO software – harder
- Experienced OO designers make good design
- New designers tend to fall back on non-OO techniques used before
- Experienced designers know something – what is it?

Why Design Patterns?

- Expert designers know not to solve every problem from first principles.
- They reuse solutions.
- These patterns make OO designs more flexible, elegant, and ultimately reusable.

Two Major Principles of OOD

- Program to an interface, not an implementation.
- Favor object compositions over class inheritance.

OO systems

- OO systems exhibit recurring structures that promote
  - find pertinent objects and factor them into classes at the right granularity.
  - define class interfaces and inheritance hierarchies
  - Establish key relationship among classes.

Introduction

- Building Architecture pattern
  - Christopher Alexander, architect
    - A Pattern Language---Towns, Buildings, Construction (1977)
    - “Each pattern describes a *problem* which occurs over and over again in our environment, and then describes the core of the *solution* to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

- Other patterns:
  - novels (tragic, romantic, crime),
  - Poem
  - movies,
  - bureaucratic memos,
  - political speeches

Patterns in engineering

- How do other engineers find and use patterns?
  - Automobile designers
    - don't design cars from scratch using the laws of physics
    - Instead, they reuse standard designs with successful track records, learning from experience
- Should software engineers make use of patterns? Why?
    - Developing software from scratch is also expensive
    - Patterns support reuse of software architecture and design

Design Patterns

- A pattern is a recurring solution to a standard problem, in a context.
- Abstracts a recurring design structure
- Comprises class and/or object
  - Dependencies
  - Structures
  - Interactions
  - Conventions

Design Patterns are NOT

- Design Patterns are NOT
  - Data structures that can be encoded in classes and reused *as is* (i.e. linked lists, hash tables)
  - Complex domain-specific designs (for an entire application or subsystem)
  - If they are not familiar data structures or complex domain-specific subsystems, *what are they*?
- They are:
  - "Descriptions of communicating objects and classes that are customized to solve a general design problemin a particular context."

Essential Elements

- Pattern name
- Problem
- Solution
- Consequences

Pattern Name

- A handle used to describe:
  - a design problem
  - its solutions
  - its consequences
- Increases design vocabulary
- Makes it possible to design at a higher level of abstraction
- Enhances communication

Problem

- Describes when to apply the pattern
- Explains the problem and its context
- May describe specific design problems and/or object structures
- May contain a list of preconditions that must be met before it makes sense to apply the pattern

Solution

- Describes the elements that make up the
  - design
  - relationships
  - responsibilities
  - collaborations
- Does not describe specific concrete implementation
- Abstract description of design problems and how the pattern solves it

Consequences

- Results and trade-offs of applying the pattern
- Critical for:
  - evaluating design alternatives
  - understanding costs
  - understanding benefits of applying the pattern
- Includes the impacts of a pattern on a system's:
  - flexibility
  - extensibility
  - portability

Description of a Design Pattern

- Describe a recurring design structure
  - Defines a common vocabulary
  - Abstracts from concrete designs
  - Identifies classes, collaborations, and responsibilities
  - Describes applicability, trade-offs, and consequences

Description of a Design Pattern

- Description of communicating objects and classes that are customized to solve a general design problem in a particular context.
- Language- & implementation-independent
- A “micro-architecture”
- Adjunct to existing methodologies (RUP, Fusion, SCRUM, etc.)

Description of a Design Pattern

- Graphical notation is generally not sufficient
- In order to reuse design decisions the alternatives and trade-offs that led to the decisions are critical knowledge
- Concrete examples are also important
- The history of the why, when, and how set the stage for the context of usage

Description of a Design Pattern

- Design patterns are described using a consistent format.
- Each pattern is devided into sections according to the template.
- Template lends a uniform structure to the information.
- It makes design patterns easiers to learn, compare, and use.

Design Patterns- Template

- Pattern Name and Classification
  - A descriptive and unique name that helps in identifying and referring to the pattern.
- Intent
  - A description of the goal behind the pattern and the reason for using it.
  - Also Known As:
  - Other names for the pattern.
- Motivation (Forces):
  - A scenario consisting of a problem and a context in which this pattern can be used.

Design Patterns- Template

- Applicability
  - Situations in which this pattern is usable; the context for the pattern.
- Structure
  - A graphical representation of the pattern. Class diagrams and Interaction diagrams may be used for this purpose.
- Participants
  - A listing of the classes and objects used in the pattern and their roles in the design.

Design Patterns- Template

continued..

- Collaboration
  - A description of how classes and objects used in the pattern interact with each other.
- Consequences
  - A description of the results, side effects, and trade offs caused by using the pattern
- Implementation
  - A description of an implementation of the pattern; the solution part of the pattern.

Design Patterns- Template

- Sample Code
  - An illustration of how the pattern can be used in a programming language.
- Known Uses
  - Examples of real usages of the pattern.
- Related Patterns
  - Other patterns that have some relationship with the pattern; discussion of the differences between the pattern and similar patterns.

Catalog of Deign Patterns

- Catalog contains 23 design patterns
- The catalog is organized according to classification of design patterns
  - Purpose
    - Creational, structural, behavioral
  - Scope
    - Class, Object

Catalog of Deign Patterns

		Purpose		
		Creational (5)	Structural(7	Behavioral(11)
Scope	Class	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory	Adapter	Chain of Responsibility
		Builder	Bridge	Command
		Prototype	Composite	Iterator
		Singleton	Decorator	Mediator
			Façade	Memento
			Flyweight	Observer
			Proxy	State
				Strategy
				Visitor

Types of Patterns

- Creational patterns:
  - Deal with initializing and configuring classes and objects
- Structural patterns:
  - Deal with decoupling interface and implementation of classes and objects
  - Composition of classes or objects
- Behavioral patterns:
  - Deal with dynamic interactions among societies of classes and objects
  - How they distribute responsibility

Types of Patterns

Creational	Structural	Behavioral
Factory Method	Adapter	Interpreter Template Method
Abstract Factory	Bridge	Chain of Responsibility
Builder	Composite	Command
Prototype	Decorator	Iterator
Singleton	Façade	Mediator
	Flyweight	Memento
	Proxy	Observer
		State
		Strategy
		Visitor

Creational Patterns

- Abstract Factory:
  - Factory for building related objects
- Builder:
  - Factory for building complex objects incrementally
- Factory Method:
  - Method in a derived class creates associates
- Prototype:
  - Factory for cloning new instances from a prototype
- Singleton:
  - Factory for a singular (sole) instance

Structural Patterns

- Adapter:
  - Translator adapts a server interface for a client
- Bridge:
  - Abstraction for binding one of many implementations
- Composite:
  - Structure for building recursive aggregations

Structural Patterns

- Decorator:
  - Decorator extends an object transparently
- Facade:
  - Simplifies the interface for a subsystem
- Flyweight:
  - Many fine-grained objects shared efficiently.
- Proxy:
  - One object approximates another

Behavioral Patterns

- Chain of Responsibility:
  - Request delegated to the responsible service provider
- Command:
  - Request is first-class object
- Iterator:
  - Aggregate elements are accessed sequentially
- Interpreter:
  - Language interpreter for a small grammar

Behavioral Patterns

- Mediator:
  - Coordinates interactions between its associates
- Memento:
  - Snapshot captures and restores object states privately
- Observer:
  - Dependents update automatically when subject changes
- State:
  - Object whose behavior depends on its state

Behavioral Patterns

- Strategy:
  - Abstraction for selecting one of many algorithms
- Template Method:
  - Algorithm with some steps supplied by a derived class
- Visitor:
  - Operations applied to elements of a heterogeneous object structure

Categorization Terms

- Scope is the domain over which a pattern applies
  - Class Scope: relationships between base classes and their subclasses (static semantics)
  - Object Scope: relationships between peer objects
- Some patterns apply to both scopes

Class::Creational

- Abstracts how objects are instantiated
- Hides specifics of the creation process
- May want to delay specifying a class name explicitly when instantiating an object
- Just want a specific protocol

Class:: Structural

- Use inheritance to compose protocols or code

- Example:
  - ***Adapter Pattern:*** makes one interface (Adaptee's) conform to another

Class:: Behavioral

- Captures how classes cooperate with their subclasses to satisfy semantics.

- Example:
  - ***Template Method***: defines algorithms step by step.

Object Scope

- Object Patterns all apply various forms of non-recursive object composition.
- Object Composition: most powerful form of reuse
- Reuse of a collection of objects is better achieved through variations of their composition, rather than through sub classing.

Object:: Creational

- Abstracts how sets of objects are created

- Example:
  - Abstract Factory: create “product” objects through generic interface

Object:: Structural

- Describe ways to assemble objects to realize new functionality
  - Added flexibility inherent in object composition due to ability to change composition at run-time
  - not possible with static class composition
- Example:
  - ***Proxy***: acts as convenient surrogate or placeholder for another object.

Object:: Behavioral

- Describes how a group of peer objects cooperate to perform a task that can be carried out by itself.

- Example:
  - Strategy Pattern: objectifies an algorithm

Benefits of Design Patterns

- Design patterns enable large-scale reuse of software architectures and also help document systems
- Patterns explicitly capture expert knowledge and design tradeoffs and make it more widely available
- Patterns help improve developer communication
- Pattern names form a common vocabulary

How Design Patterns Solve Design Problems

- The problems
  - Finding Appropriate Objects
  - Determining Object Granularity
  - Specifying Object Interfaces
  - Specifying Object Implementations
  - Putting Reuse Mechanisms to Work
  - Relating Run-Time and Compile-Time Structures.
  - Designing for Change

Finding Appropriate Objects

- To decomposing a system, we must consider:
  - encapsulation, granularity, dependency, flexibility, performance, evolution, reusability, …
- Strict modeling of the real world may lead to inflexibility.
- The abstractions that emerge during design are key to making a design flexible.
- Design patterns help you identify less-obvious abstractions and the objects that can capture them

Determining Object Granularity

- Objects can vary tremendously in size and number.
- Design patterns address this issue as well.
- The patterns: Facade, Flyweight, AbstractFactory, Builder, Visitor, Command …

Specifying Object Interface

- Interface are fundamental in OO system.
  - signature, type, supertype, dynamic binding, polymorphism.
- Design patterns help you define interfaces by identifying their key elements and the kind of data that get sent across an interface.
- A pattern also tell you what not to put.
- Memento, Decorator, Proxy, Visitor

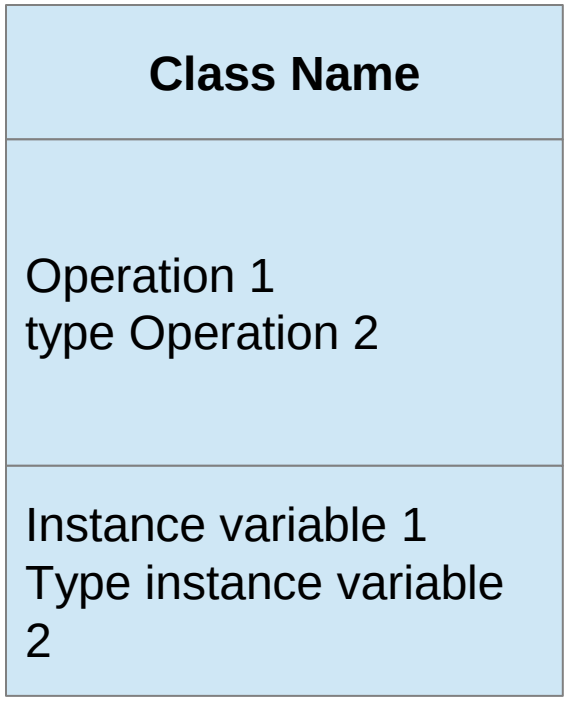
Specifying Object Interface

- Every operation has a name, return type and parameters : known as signature of operations
- Set of all signatures defined by an object's operations is known as Interface to the object
- There can be different objects having same interfaces
- Example : Methods in Duck, MallardDuck and RedHeadDuck
- Allows to use polymorphism (dynamic binding)

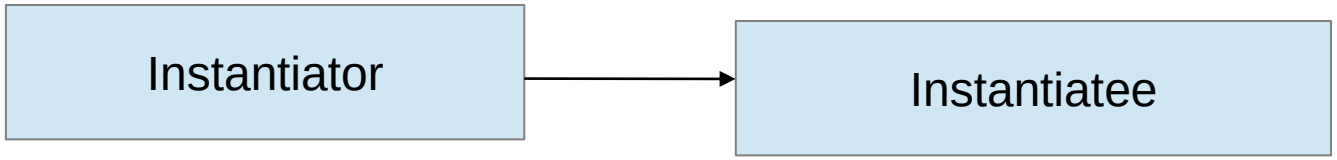
Specifying Object Implementation

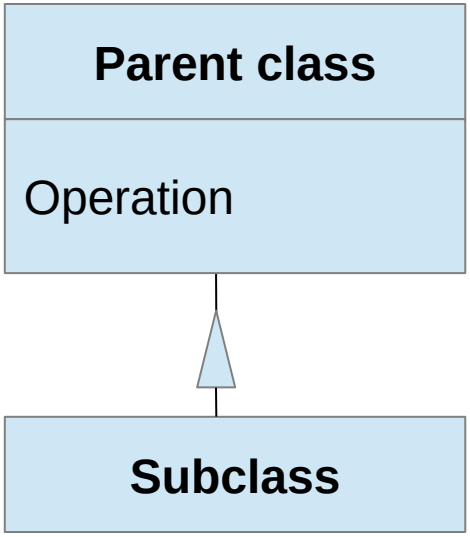
- About the implementation:
  - Object's implementation is defined by class.
  - Objects are created by instantiating a class.
  - New classes can be defined by class inheritance. (subclass and parent class).
  - abstract class and concrete classes.
  - A class may override an operation in the parent class.
  - Mixin class is a class that's intended to provide an optional interface or functionality.

Class representation

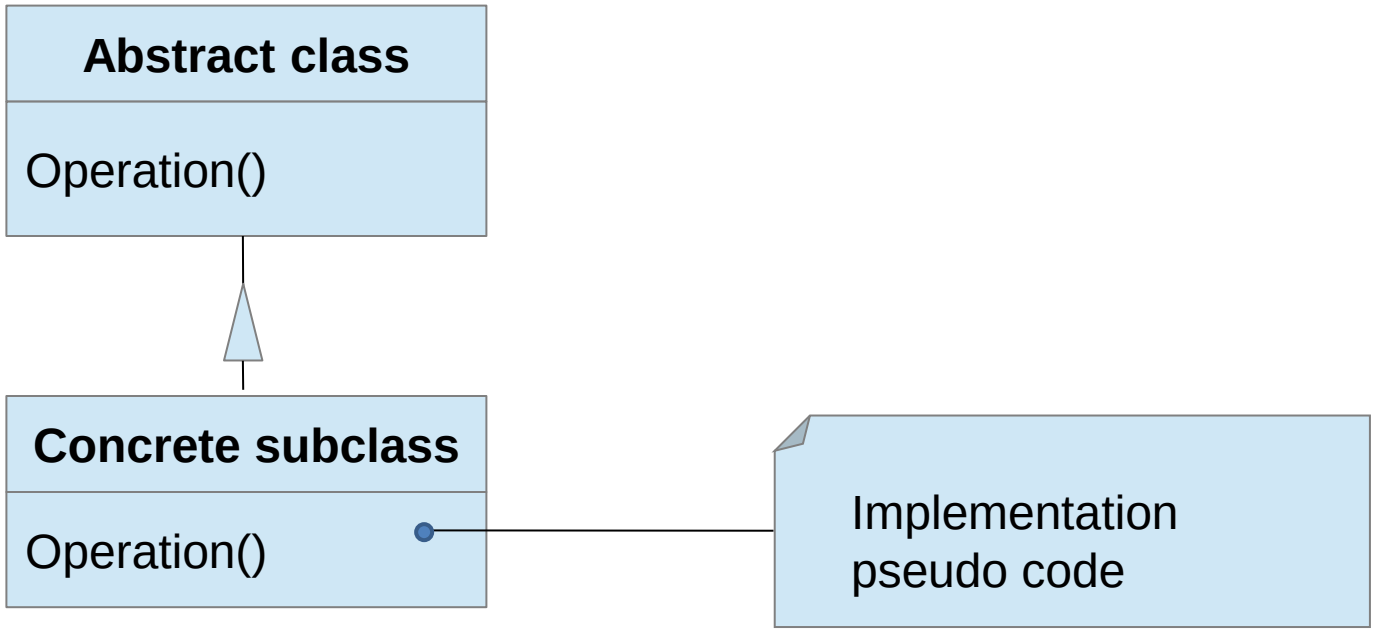


Instantiation and Inheritance

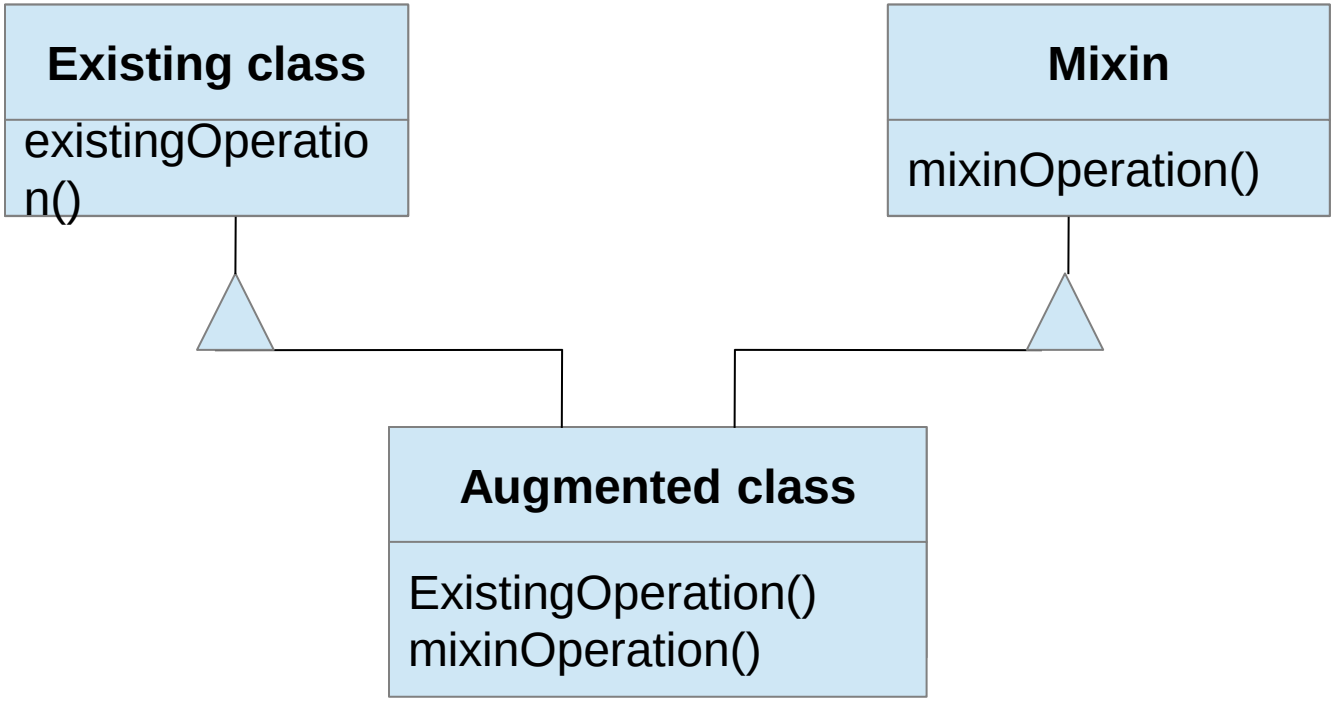




Representation of abstract class



Mixin Class


Existing class
existingOperation()
Mixin
mixinOperation()
Augmented class
ExistingOperation()
mixinOperation()

Specifying Object Implementation

- The issues:
  - Class versus Interface Inheritance.
  - Programming to an Interface, not an implementation

Class versus Interface Inheritance

- The difference between class and type：
  - class defines the implementation.
  - type refers to the interface.
  - An object can have many types, and objects of different classes can have the same type.
- In some languages like C++, Eiffel, classes specify both an object's type and its implementation.

Class versus Interface Inheritance

- Class inheritance versus interface inheritance
  - class inheritance defines an object's implementation in terms of another object's implementation. (code and representation sharing).
  - interface inheritance define when an object can be used in place of another. (subtyping)

Class versus Interface Inheritance

- Though most programming language don't support the distinction between interface and implementation inheritance, people should make the distinction in practice.
- Related patterns: Chain of Responsibility, Composite, Command, Observer, State, Strategy.

Programming to an interface, not an Implementation

- Implementation reuse is only half of the purpose of inheritance.
- Two benefits manipulating objects solely in terms of interface:
  - Clients remain unaware of the specific types of objects they use.
  - Clients remain unaware of the classes implementing the objects.

Programming to an interface, not an Implementation

- Declare variables using an interface defined by an abstract class.
- Instantiate concrete classes somewhere.
- The creational patterns make the instantiation possible.
  - Abstract Factory, Builder, Factory Method, Prototype, Singleton.

Putting Reuse Mechanisms to Work

- Inheritance versus Composition
- Delegation
- Inheritance versus Parameterized Types

Inheritance versus Composition

- Inheritance is referred to as white-box reuse.
- Object composition: assembling or composing objects to get more complex objects. referred as black-box reuse.

Reuse by Inheritance

- Inheritance advantages
  - Defined statically at compile time
  - Supported by programming languages so straightforward to use
  - Encourages reuse and easy modification of inherited methods
- Disadvantages
  - Cannot change the implementations inherited from parent classes at run-time, because inheritance is defined at compile time
  - Inheritance breaks encapsulation
  - Any change in parent class implementation causes the subclass to change

Reuse by Composition

- Composition requires objects to respect each others' interfaces. So we don't break encapsulation.
- Any object can be replaced at run-time by another as long as it has the same type.
- Reduce the implementation dependencies.
- The classes and class hierarchies will remain small.

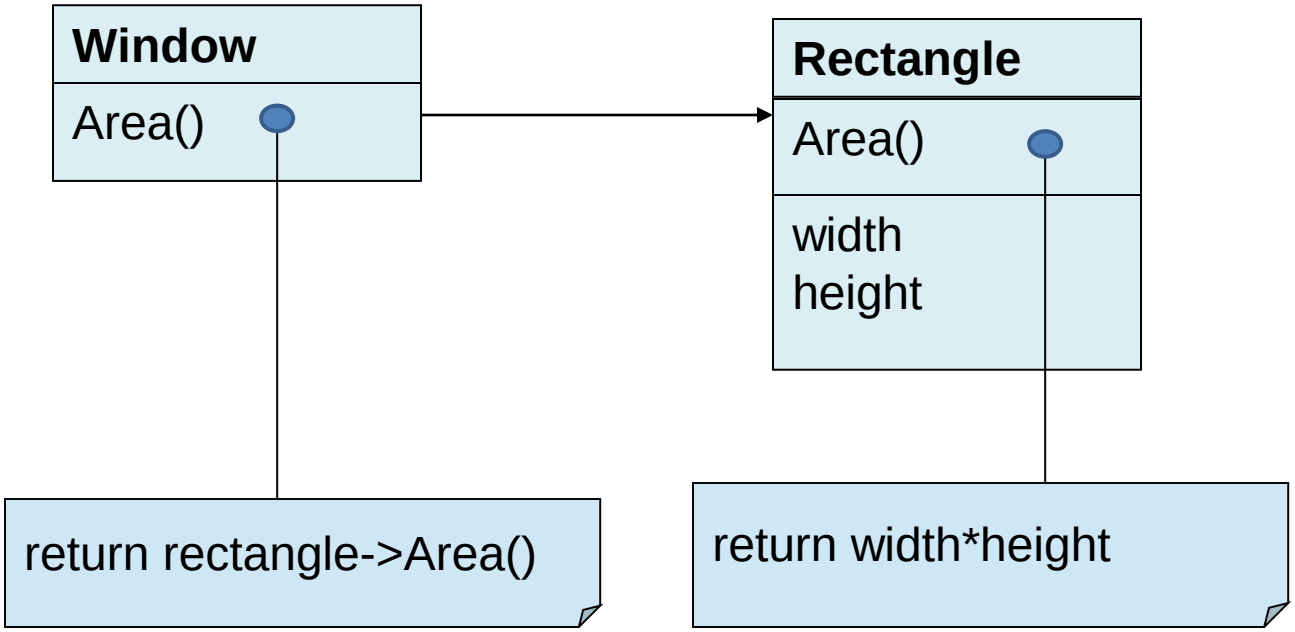
Favor object composition over class inheritance

- You should be able to get all the functionality you need by assembling existing components.
- You may use inheritance to define new components if the existing components are not enough.
- Designs are often made more reusable by depending more on object composition.

Delegation

- Two objects are involved in handling a request : A receiving object delegates operations to its Delegate
- Example : Instead of saying a MallardDuck flies and quacks we say that A MallardDuck has FlyingBehavior and QuackingBehavior

Example


Window
Area()
Rectangle
Area()
width
height
return rectangle->Area()
return width*height

Delegation

- The advantage:
  - easy to compose behaviors at run-time
  - easy to change the way they're composed.
- Disadvantage:
  - Dynamic, highly parameterized software is harder to understand than static software.
  - run-time inefficiencies.
- A good choice when it simplifiers more than it complicates.

Inheritance verses parameterized type

- Parameterized type can be used to reuse a functionality same as inheritance
- Implemented with the help of templates in C++ and generics in Java
- Used by some design patterns

Compare the three reuse methods

- Object composition lets you change the behavior being composed at run-time. It requires indirection and can be less efficient.
- Inheritance lets you provide default implementation, and lets you override them.
- Parameterized types let you change the types that a class can use.

Relating compile time and runtime structures

- Association and aggregation represent has-a relationship
- Runtime structures are different
- Aggregation- lifetimes of aggregator and aggregate are same
- Association - lifetimes of aggregator and aggregate are different

How to select a Design Pattern

- Consider how design patterns solve design problems
- Scan Intent section
- Study how patterns interrelate
- Study patterns of like purpose
- Examine the causes of redesign
- Consider what should be variable in your design
  - Consider what you need to change without redesign
  - Learn at a later stage

Aggregation and Acquaintance

- Aggregation: one object owns or is responsible for another object.
- Acquaintance: an object merely knows of another object. (also called ‘association’ or ‘using’)
- Aggregation and acquaintance are similar. They are determined more by intent than by language mechanisms.
- They are significantly different at the run-time

Aggregation

Designing for change

- A system should be designed anticipating future changes
- It should be robust to changes
- Design patterns help us to design such systems

Common causes of redesign

- Creating an object by specifying a class explicitly.
  - Abstract Factory, Factory Method, Prototype.
- Dependence on specific operations:
  - Chain of Responsibility, Command.
- Dependence on hardware and software platform
  - Chain of Responsibility, Command

Common causes of redesign

- Dependence on object representations or implementations
  - Abstract Factory, Bridge, Memento, Proxy.
- Algorithm dependencies
  - Builder, Iterator, Strategy, Template Method, Visitor.
- Tight coupling
  - Abstract Factory, Bridge, Chain of Responsibility, Command, Facade, Mediator, Observer.

Common causes of redesign

- Extending functionality by subclassing
  - Bridge, Chain of Responsibility, Composite, Decorator, Observer, Strategy.
- Inability to alter conveniently
  - Adapter, Decorator, Visitor

Application Programs

- Design patterns help to make applications more maintainable and extendible
- Toolkits – Libraries that provide general purpose functionalities, ex. iostream in C++ : Should be general enough to be used in any application : Design patterns help in toolkit development

Toolkits

- Toolkits is a set of related and reusable classes designed to provide useful, general-purpose functionality.
- Toolkits emphasize code reuse.
- The designer should avoid assumptions and dependencies that can limit the toolkit's flexibility.

Frameworks

- Frameworks contain a set of cooperating classes that make up a reusable design for specific class of software
- Ex. A framework to build compilers for different languages

Patterns vs Frameworks

- Patterns are lower-level than frameworks
- Frameworks typically employ many patterns:
  - Factory
  - Strategy
  - Composite
  - Observer
- Done well, patterns are the “plumbing” of a framework

How to select a design pattern

- Approaches to finding the design patterns:
  - Consider how design patterns solve design problems.
  - Scan intent sections.
  - Study how patterns interrelate.
  - Study patterns of like purpose.
  - Examine a cause of redesign.
  - Consider what should be variable in your design.(see table 1.2)

Aspects that can vary

Purpose	Design Pattern	Aspects that can vary
Creational	Abstract Factory	Families of product objects
	Builder	How a composite object gets created
	Factory Method	Subclass of object that is instantiated
	Prototype	Class of object that is instantiated
	Singleton	The sole instance of a class

Aspects that can vary

Purpose	Design Pattern	Aspects that can vary
Structural	Adapter	Interface to an object
	Bridge	Implementation of an object
	Composite	Structure and composition of object
	Decorator	Responsibilities of an object without subclassing
	Façade	Interface to a subsystem
	Flyweight	Storage costs of object
	Proxy	How an object is accessed; its location

Aspects that can vary

Purpose	Design Pattern	Aspects that can vary
Behavioral	Chain of Responsibility	Object that can fulfill a request
	Command	When and how a request is fulfilled
	Interpreter	Grammar and interpretation of a language
	Iterator	how an aggregates elements are accessed, traversed
	Mediator	How and which objects interact with each other

Aspects that can vary

Purpose	Design Pattern	Aspects that can vary
Behavioral	memento	What private information is stored outside an object
	observer	Number of objects that depend on another object
	state	States of an object
	strategy	algorithm
	template	Steps of algorithm
	visitor	Operations that can be applied to objects without changing their case

How to use a Design Pattern

- Read the pattern once through for an overview.
- Go back and study the Structure, Participants, and Collaborations sections.
- Look at the sample Code section to see a concrete example of the pattern in Code.
- Choose names for pattern participants that are meaningful in the application context.

How to use a Design Pattern

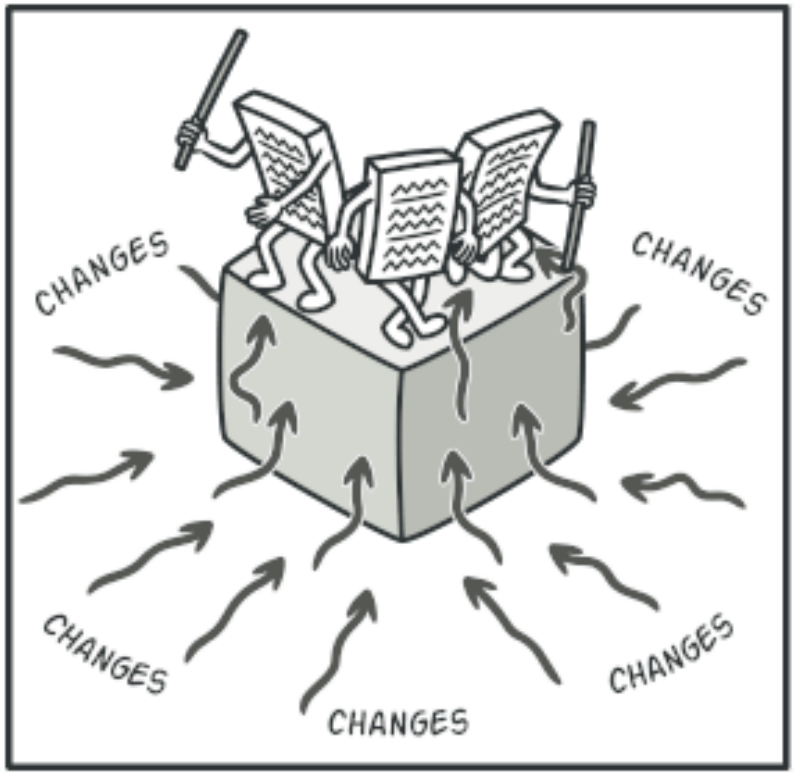
- Define the classes.
- Define application-specific names for operations in the pattern.
- Implement the operations to carry out the responsibilities and collaborations in the pattern

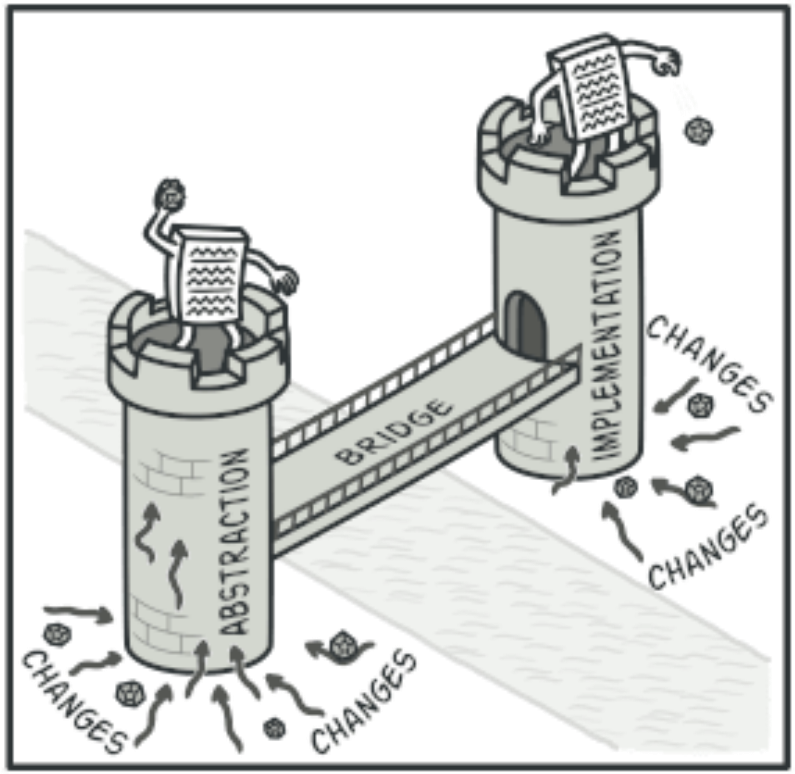
Benefits of Design Patterns

- Design patterns enable large-scale reuse of software architectures and also help document systems
- Patterns explicitly capture expert knowledge and design tradeoffs and make it more widely available
- Patterns help improve developer communication
- Pattern names form a common vocabulary

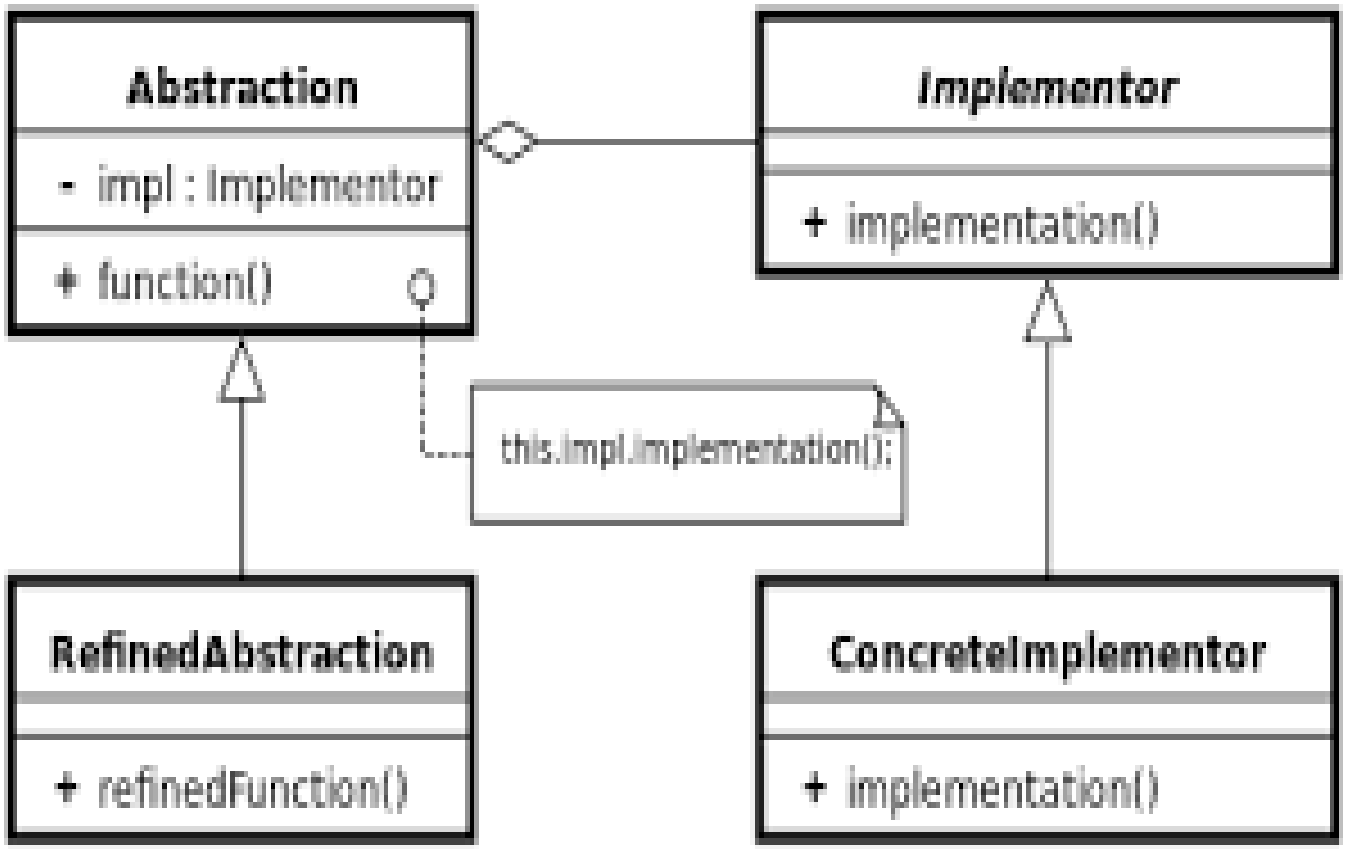
Bridge

- **Intent**
- **Bridge** is a structural design pattern that lets you split a large class or a set of closely related classes into two separate hierarchies—abstraction and implementation—which can be developed independently of each other.


CHANGES
CHANGES
CHANGES
CHANGES
CHANGES
CHANGES


ABSTRACTION
BRIDGE
IMPLEMENTATION
CHANGES
CHANGES
CHANGES
CHANGES
CHANGES

Structure


Abstraction
- impl : Implementor
+ function()
Implementor
+ implementation()
this.impl.implementation();
RefinedAbstraction
+ refinedFunction()
ConcreteImplementor
+ implementation()

THANK YOU…